Modulo 3

Programmazione procedurale vs OOP

Programmazione procedurale vs OOP

- Programmazione procedurale e orientata agli oggetti: due paradigmi di programmazione.
- Programmazione procedurale: approccio lineare e sequenziale.
- Programmazione orientata agli oggetti: approccio basato su oggetti e classi.

Programmazione procedurale vs OOP

- **Programmazione procedurale**
- Definizione e caratteristiche della programmazione sequenziale
- Analisi di problemi e algoritmi sequenziali
- Pseudocodice e flowchart
- Case studies: esempi di problemi risolti sequenzialmente
- Ottimizzazione di algoritmi sequenziali
- Debugging e identificazione degli errori
- Test e verifica di algoritmi
- Best practicese stile nella programmazione sequenziale
- Introduzione alla complessità algoritmica
- Esempi di codice sequenziale

OOP

- Introduzione alla OOP: concetti chiave
- Classi e oggetti: definizione e utilizzo
- Attributi, metodi e costruttori
- Incapsulamento e metodi privati
- Ereditarietà: classi base e derivate
- Polimorfismo e metodi sovrascritti
- Classi astratte e interfacce
- Composizione e aggregazione
- Design patternscomuni in OOP
- Proprietà e metodi speciali in Python
- OOP avanzata: metaclassese decorators

Modulo 3

Programmazione procedurale vs OOP

Cosa sono

La programmazione procedurale e la programmazione orientata agli oggetti (OOP, Object-Oriented Programming) sono due paradigmi di programmazione fondamentali, ognuno con le sue caratteristiche e modi di approccio alla scrittura del codice.

Modulo 3

Programmazione procedurale vs OOP

Programmazione Procedurale

La programmazione procedurale è un paradigma di programmazione basato su procedure o funzioni. Può essere considerata "sequenziale" nel senso che il codice viene eseguito in sequenza, ma questo è vero anche per altri paradigmi di programmazione. Le caratteristiche chiave della programmazione procedurale includono:

Focalizzata sulle Procedure/Funzioni: Il codice è organizzato in procedure o funzioni che eseguono operazioni specifiche.
Stato Condiviso: Le funzioni lavorano tipicamente su dati passati come argomenti o su dati globali.
Strutturata: Segue un approccio top-down, iniziando con un problema complesso e suddividendolo in problemi più piccoli e gestibili.

Esempi di Linguaggi: C, Fortran e BASIC sono esempi di linguaggi orientati alla programmazione procedurale.

Modulo 3

Programmazione procedurale vs OOP

Programmazione Orientata agli Oggetti (OOP)

La programmazione orientata agli oggetti è un paradigma basato sull'idea di "oggetti", che possono contenere dati (sotto forma di campi, spesso chiamati attributi) e codice (sotto forma di procedure, spesso chiamate metodi). Le caratteristiche principali dell'OOP includono:

Incapsulamento: I dati e i metodi che lavorano su questi dati sono racchiusi come unità chiamate oggetti. Questo nasconde i dettagli interni e mostra solo le funzionalità necessarie all'esterno.
Ereditarietà: Le classi possono ereditare caratteristiche da altre classi, promuovendo il riutilizzo del codice.
Polimorfismo: Gli oggetti di una superclasse possono essere trattati come oggetti di una sottoclasse, facilitando la sostituzione e la modularità.
Astrazione: Fornisce una semplice interfaccia agli oggetti complessi sottostanti e nasconde la complessità implementativa.

Esempi di Linguaggi: Java, Python e C++ sono esempi di linguaggi che supportano l'OOP.

Modulo 3

Programmazione procedurale vs OOP

Confronto e Utilizzo

Mentre la programmazione procedurale è spesso vista come più semplice e lineare, l'OOP offre un approccio più modulare e flessibile, particolarmente utile per grandi progetti e applicazioni complesse. In realtà, molti progetti moderni combinano entrambi gli approcci per sfruttare i vantaggi di entrambi.

È anche importante notare che molti linguaggi di programmazione, come Python e Java, supportano entrambi i paradigmi, permettendo agli sviluppatori di scegliere l'approccio più adatto al problema che stanno affrontando.

Modulo 3

Programmazione procedurale vs OOP

Analisi di problemi e algoritmi sequenziali

1.Definizione del Problema: Comprendere chiaramente cosa deve essere risolto. Questo richiede spesso di identificare gli input, gli output desiderati, e le condizioni o limitazioni del problema.
2.Scomposizione: Suddividere il problema in componenti più piccoli e gestibili. Questo approccio, noto come decomposizione, aiuta a semplificare il processo di soluzione.
3.Identificazione di Sotto-problemi: Riconoscere i sotto-problemi che, una volta risolti, contribuiranno alla soluzione complessiva.
4.Ricerca di Pattern e Analogie: Cercare schemi ricorrenti o analogie con problemi già risolti che possono fornire indicazioni su come affrontare il problema corrente.
5.Valutazione delle Risorse: Considerare le risorse disponibili, come tempo, memoria, e potenza di calcolo, per determinare il metmetodo più efficiente di risoluzione del problema.

Modulo 3

Programmazione procedurale vs OOP

Algoritmi Sequenziali
Una volta completata l'analisi, l'obiettivo è sviluppare un algoritmo che risolva il problema. Gli algoritmi sequenziali sono una serie di passaggi eseguiti uno dopo l'altro. Essi includono:

Sequenza: Una serie di istruzioni eseguite in ordine, una dopo l'altra.

Selezione: Uso di istruzioni condizionali (come if, else, switch) per prendere decisioni e biforcare il flusso di controllo.

Iterazione: Uso di cicli (come for, while) per ripetere un blocco di codice fino a quando una condizione specificata non è più vera.

Linearità: Ogni passo segue il precedente in modo lineare, senza complicazioni come richiami di funzioni multiple o operazioni concorrenti.

Clartà e Determinismo: Ogni passaggio è chiaro e non ambiguo, e per un dato input, l'algoritmo produce sempre lo stesso output.

Modulo 3

Programmazione procedurale vs OOP

Esempio Pratico
Per esempio, considera il problema di ordinare un elenco di numeri. Un algoritmo sequenziale per questo potrebbe essere l'ordinamento a bolle, che confronta ripetutamente elementi adiacenti e li scambia se sono nell'ordine sbagliato. Questo processo viene ripetuto fino a quando l'intero elenco non è ordinato.

Importanza nell'Insegnamento della Programmazione
Nell'insegnamento della programmazione, soprattutto ai principianti, la focalizzazione sugli algoritmi sequenziali aiuta a costruire una solida comprensione del flusso di controllo e della logica di base, che sono essenziali per affrontare problemi più complessi e per apprendere paradigmi di programmazione più avanzati come la programmazione orientata agli oggetti.

Modulo 3

Pseudocodice e flowchart

Pseudocodice

Lo pseudocodice è un metodo informale per descrivere un algoritmo. Utilizza una struttura simile a quella del codice di programmazione, ma è scritto in un linguaggio naturale. Il pseudocodice non è eseguibile su un computer; invece, serve a illustrare in modo chiaro e conciso come un algoritmo dovrebbe funzionare. Le caratteristiche principali includono:

Linguaggio Semplice: Usa termini di linguaggio naturale combinati con strutture simili a quelle del codice, rendendolo comprensibile indipendentemente dalla conoscenza di un linguaggio di programmazione specifico.
Convenzioni Flessibili: Non segue una sintassi rigorosa come i linguaggi di programmazione, permettendo flessibilità nella scrittura.
Strutture di Controllo: Include strutture di controllo comuni come cicli (for, while), condizioni (if, else), ecc.
Focus sulla Logica: Concentra l'attenzione sulla logica dell'algoritmo piuttosto che sui dettagli di implementazione specifici del linguaggio di programmazione.

Esempio di pseudocodice per un semplice algoritmo di ordinamento:X

```
procedura ordina(lista)
    per i da 0 a lunghezza(lista) - 1
        per j da 0 a lunghezza(lista) - i - 1
            se lista[j] > lista[j + 1]
                scambia lista[j] e lista[j + 1]
    fine per
fine per
fine procedura
```

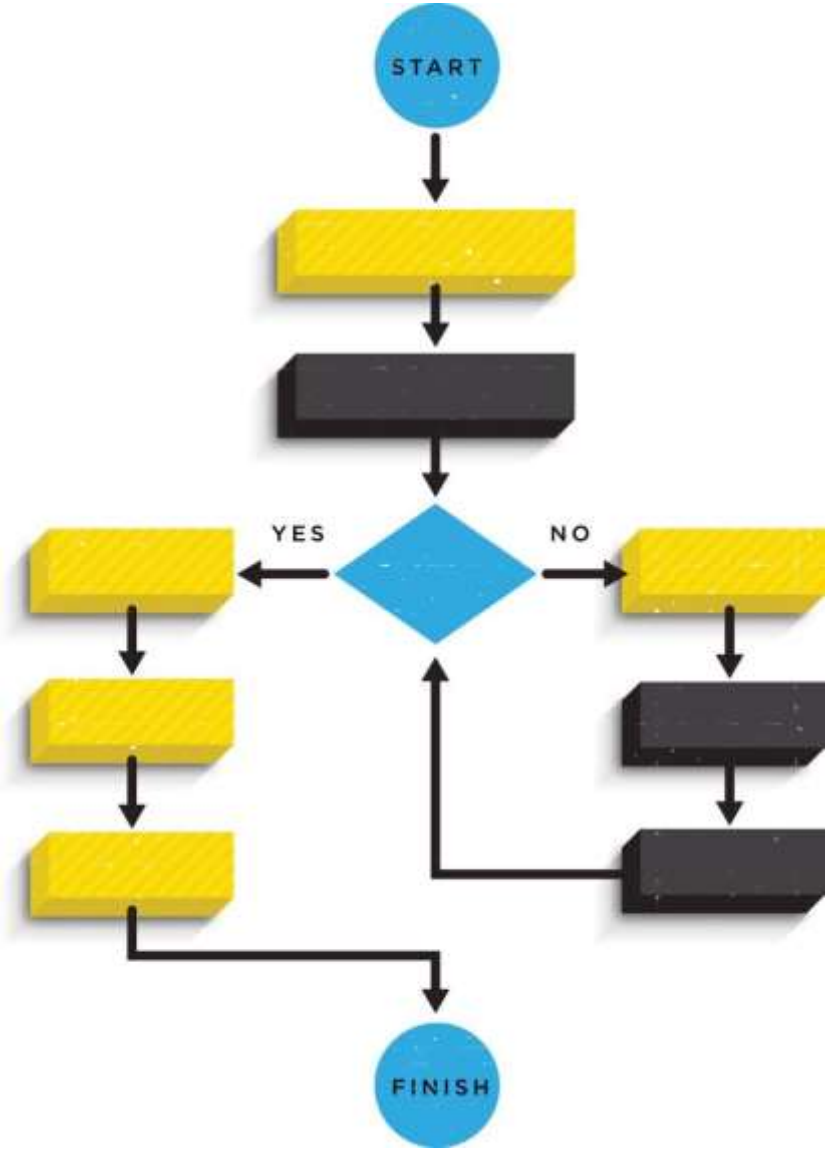
Modulo 3

Pseudocodice e flowchart

Flowchart (Diagrammi di Flusso)

I flowchart, o diagrammi di flusso, sono rappresentazioni grafiche degli algoritmi. Utilizzano simboli standardizzati per rappresentare diversi tipi di istruzioni o passaggi, con frecce che mostrano il flusso del programma. Caratteristiche principali:

Rappresentazione Visiva: Fornisce una rappresentazione visuale della sequenza di passaggi e delle decisioni all'interno di un algoritmo.
Simboli Standardizzati: Include rettangoli per le istruzioni, rombi per le decisioni, ovali per l'inizio e la fine, tra gli altri.
Facilità di Comprensione: Utile per comprendere e comunicare il flusso di controllo in un algoritmo, specialmente in team multidisciplinari.
Utilità nella Progettazione e nel Debugging: Aiuta nella pianificazione della struttura di un programma e nella risoluzione dei problemi.
Un flowchart per lo stesso algoritmo di ordinamento potrebbe mostrare il processo di iterazione attraverso la lista, la verifica delle condizioni e lo scambio di elementi con simboli e connettori che indicano il flusso di esecuzione.



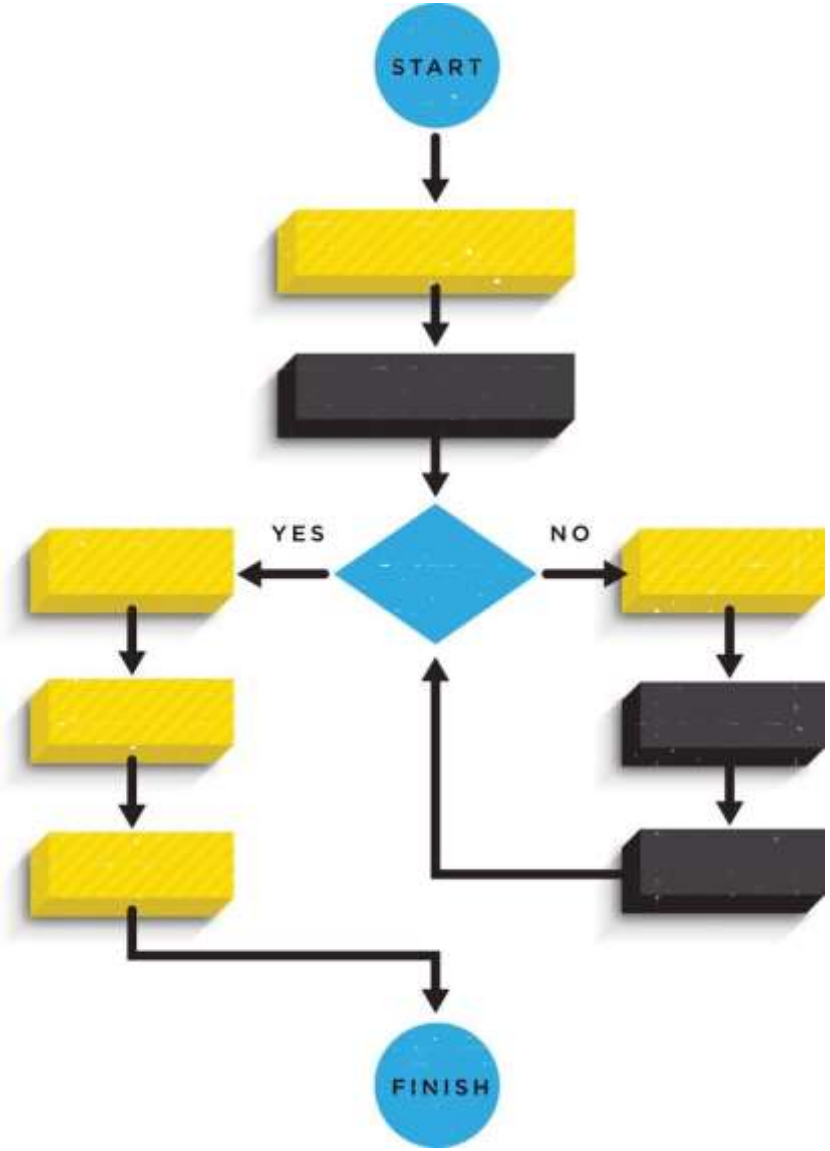
Modulo 3

Pseudocodice e flowchart

Flowchart (Diagrammi di Flusso)

Utilizzo

Pseudocodice e flowchart sono spesso utilizzati in combinazione durante la fase di progettazione del software. Mentre il pseudocodice è eccellente per dettagliare la logica in un formato simile al codice, i flowchart sono utili per visualizzare il flusso di controllo e le decisioni. Entrambi sono strumenti didattici efficaci e sono ampiamente utilizzati nell'istruzione di informatica per aiutare gli studenti a capire e progettare algoritmi prima di implementarli in un linguaggio di programmazione specifico.



Modulo 3

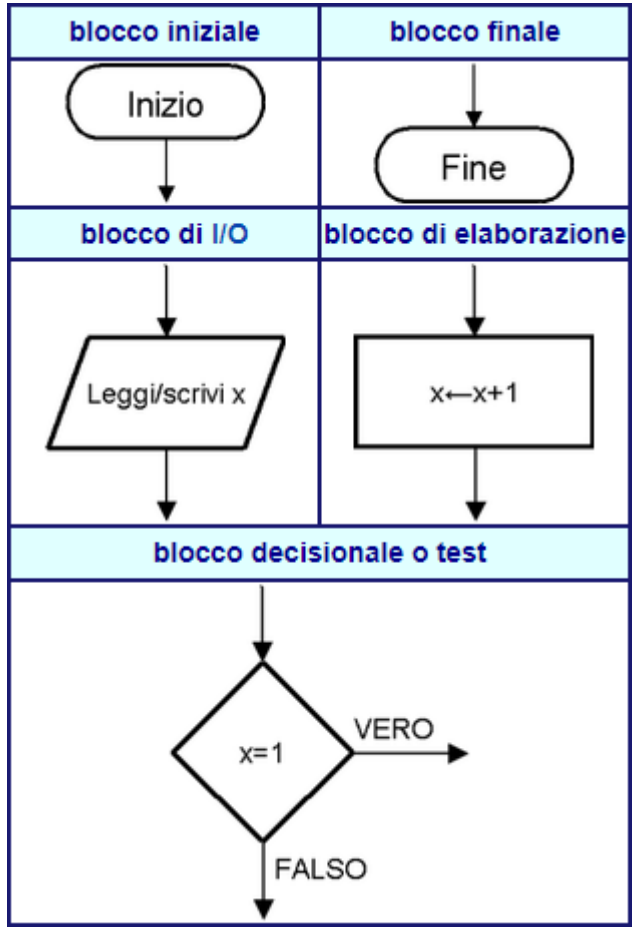
Pseudocodice e flowchart

Flowchart :: Blocchi elementari

Esistono varie notazioni per la rappresentazione con diagrammi di flusso.Tutte le notazioni sottendono a un meta-modello molto semplice, caratterizzato da una lettura sequenziale:

si parte dal blocco iniziale
si segue la freccia in uscita
si giunge al blocco successivo e si effettua l'operazione descritta nel blocco
si procede iterando i passi 2 e 3 fino a giungere al blocco finale.
Tra le operazioni si distinguono:

azioni, che comportano un'attività o un'elaborazione da svolgere
test, che indicano due o più direzioni in base a un fattore di decisione (vero o falso)
ingresso/uscita, che comportano l'immissione di informazioni dall'esterno oppure l'invio di informazioni verso l'esterno
La notazione più semplice e più frequentemente utilizzata prevede dunque 5 tipi di blocchi elementari:



Modulo 3

Pseudocodice e flowchart

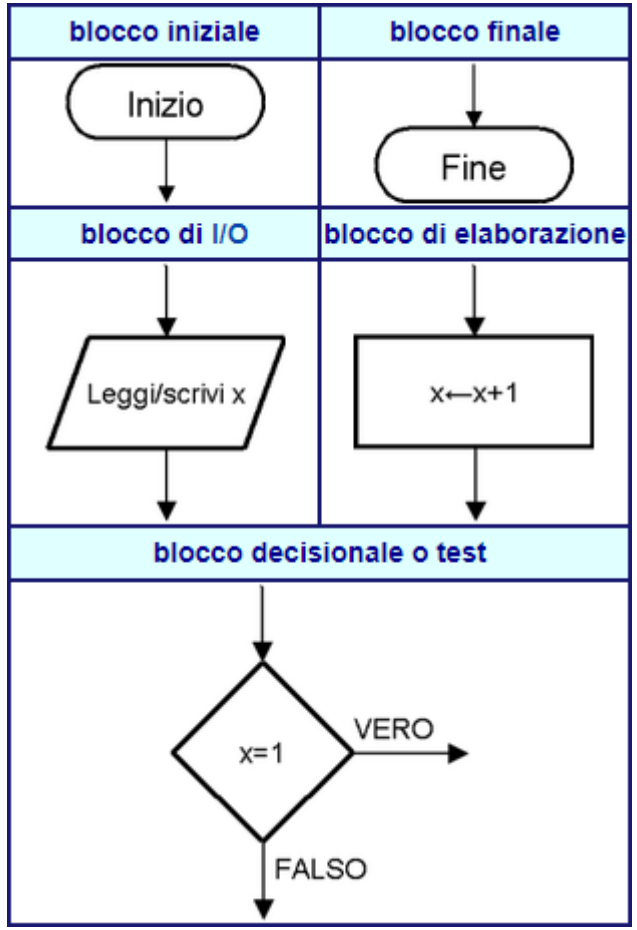
Una combinazione di blocchi elementari descrive un algoritmo se:
•viene usato un numero finito di blocchi
•lo schema inizia con un *blocco iniziale* e termina con un *blocco finale*
•ogni blocco soddisfa le *condizioni di validità*

Condizioni di validità

- condizioni sui blocchi:
  - *blocco azione* e *blocco lettura/scrittura*: ogni blocco di questi due tipi ha una sola freccia entrante e una sola freccia uscente
  - *blocco decisionale*: ogni blocco di questo genere ha una sola freccia entrante e due frecce uscenti

•condizioni sulle frecce:
  - ogni *freccia* deve entrare in un blocco

•condizioni sui percorsi:
  - dal *blocco iniziale* deve essere possibile raggiungere ogni blocco
  - da ogni blocco dev'essere possibile raggiungere il *blocco finale*



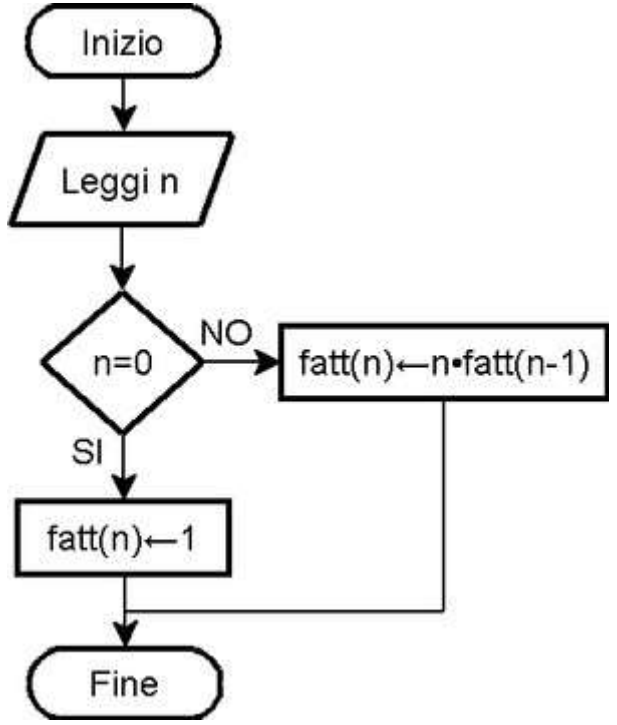
Modulo 3

Pseudocodice e flowchart

Dato l'algoritmo per il calcolo del fattoriale in maniera ricorsiva

$$\text{fatt}(n) = n! = \begin{cases} 1, & \text{se } n = 0 \\ n \cdot (n-1)!, & \text{se } n \neq 0 \end{cases}$$

un possibile diagramma di flusso per rappresentarlo:



Dove tale diagramma rappresenta una procedura ricorsiva, denominata "fatt", che richiama sé stessa.

Modulo 3

Caso di studio (esempio)

Calcolo della Media

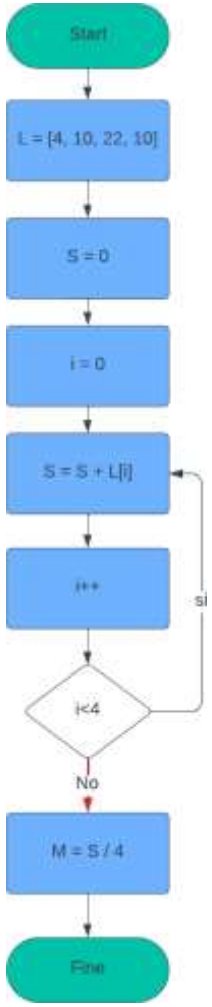
Problema: Calcolare la media di un insieme di numeri.
Soluzione Sequenziale:
1. Sommare tutti i numeri nella lista.
2. Contare il numero di elementi nella lista.
3. Dividere la somma totale per il numero di elementi.
4. Restituire il risultato.

Modulo 3

Caso di studio (esempio)

Calcolo della Media

Problema: Calcolare la media di un insieme di numeri.
Soluzione Sequenziale:
1. Sommare tutti i numeri nella lista.
2. Contare il numero di elementi nella lista.
3. Dividere la somma totale per il numero di elementi.
4. Restituire il risultato.

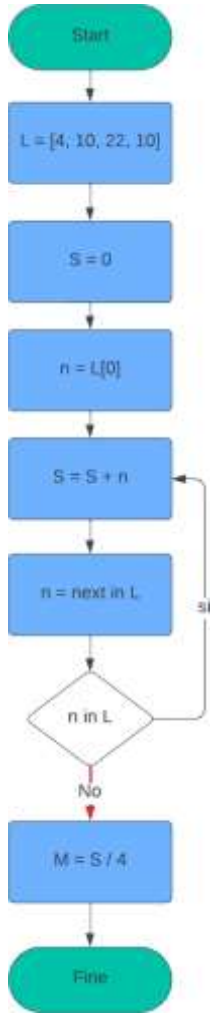


Modulo 3

Caso di studio (esempio)

Calcolo della Media

In Python si usa anche far semplicemente scorrere i valori di una sequenza associandoli direttamente ad una variabile uno ad uno fino a quando la sequenza non è finita



bitCamp

Modulo 3

Caso di studio (esempio)

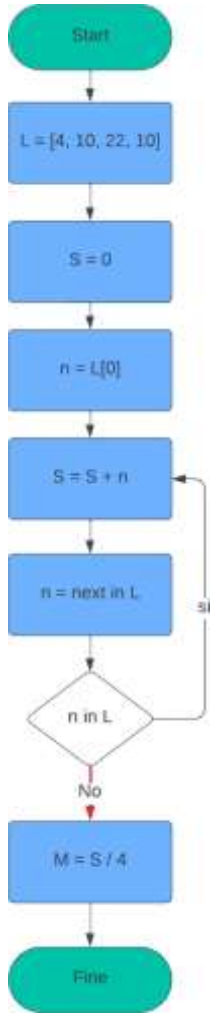
Calcolo della Media

Una possibile implementazione pratica del flow chart in una funzione

```
def calcola_media(numeri):
    if len(numeri) == 0:
        return "Errore: l'elenco è vuoto."

    somma = sum(numeri)
    numero_elementi = len(numeri)
    media = somma / numero_elementi
    return media

# Esempio di utilizzo
numeri = [10, 20, 30, 40, 50]
media = calcola_media(numeri)
print(f"La media dei numeri è: {media}")
```



Modulo 3

Ottimizzazione di algoritmi sequenziali

L'ottimizzazione di algoritmi sequenziali è un aspetto cruciale della programmazione, specialmente quando si tratta di migliorare le prestazioni e l'efficienza del codice. L'obiettivo è rendere l'algoritmo più rapido, meno dispendioso in termini di risorse e più efficace nel risolvere il problema. Ecco alcuni aspetti chiave e tecniche per ottimizzare algoritmi sequenziali:

Modulo 3

Ottimizzazione di algoritmi sequenziali

Riduzione della Complessità

•**Analisi della Complessità:** Capire la complessità temporale (quanto tempo impiega un algoritmo a eseguire) e spaziale (quanta memoria utilizza) dell'algoritmo. Utilizzare l'analisi Big O per valutare l'efficienza.

•**Sostituzione di Algoritmi:** Sostituire algoritmi meno efficienti con altri più efficienti. Ad esempio, sostituire l'ordinamento a bolle con l'ordinamento QuickSort o MergeSort.

Modulo 3

Ottimizzazione di algoritmi sequenziali

Utilizzo Efficace delle Strutture Dati

- **Scelta delle Strutture Dati:** Usare strutture dati appropriate per migliorare l'efficienza. Ad esempio, l'utilizzo di hashtable per operazioni di ricerca veloci.
- **Gestione della Memoria:** Ridurre l'uso della memoria dove possibile, come evitare la duplicazione dei dati.

Modulo 3

Ottimizzazione di algoritmi sequenziali

Ottimizzazione del Codice

- **Riduzione delle Operazioni:** Rimuovere operazioni non necessarie o ridondanti all'interno dei cicli.
- **Uso di Tecniche di Programmazione:** Come la memoizzazione o la programmazione dinamica per problemi che hanno sovrapposizioni di sotto-problemi e strutture ottimali di sotto-struttura.

Modulo 3

Ottimizzazione di algoritmi sequenziali

Sfruttare il Contesto Specifico

- **Assunzioni sul Set di Dati:** Se ci sono informazioni specifiche sul set di dati, queste possono essere utilizzate per ottimizzare l'algoritmo. Ad esempio, se i dati sono parzialmente ordinati, potrebbe essere possibile usare algoritmi più efficienti.
- **Parallelizzazione e Concorrenza:** Per i sistemi che lo supportano, dividere il lavoro in parti che possono essere eseguite in parallelo.

Modulo 3

Ottimizzazione di algoritmi sequenziali

Testing e Profiling

•**Profiling dell'Algoritmo:** Utilizzare strumenti di profiling per identificare colli di bottiglia nel codice.

•**Test di Regressione:** Assicurarsi che le ottimizzazioni non introducano bug o alterino l'output atteso.

Ricerca Continua e Aggiornamenti

•**Mantenere Aggiornato il Codice:** Con l'evoluzione continua delle tecniche di programmazione e delle piattaforme hardware, è importante aggiornare regolarmente gli algoritmi.

•**Ricerca di Nuove Tecniche:** Essere aggiornati sulle ultime ricerche e innovazioni nel campo degli algoritmi.

L'ottimizzazione degli algoritmi è un processo iterativo e spesso richiede un bilanciamento tra diversi fattori, come tempo di esecuzione, utilizzo della memoria e complessità del codice. È importante valutare le esigenze specifiche dell'applicazione e del contesto per scegliere la strategia di ottimizzazione più appropriata.

Modulo 3

Ottimizzazione di algoritmi sequenziali

L'ottimizzazione di algoritmi sequenziali è un processo fondamentale per migliorare le prestazioni e l'efficienza del codice, soprattutto in contesti dove il tempo di esecuzione e l'uso delle risorse sono critiche. L'obiettivo è rendere l'algoritmo più veloce, meno dispendioso in termini di risorse e più efficace nel risolvere il problema proposto.

Modulo 3

Ottimizzazione di algoritmi sequenziali

Riduzione della Complessità Computazionale

•**Analisi della Complessità:** Comprendere la complessità temporale e spaziale dell'algoritmo. L'uso dell'analisi Big O può aiutare a identificare e migliorare le parti inefficienti dell'algoritmo.

•**Sostituzione con Algoritmi più Efficienti:** In alcuni casi, un algoritmo può essere sostituito con un altro che ha una complessità temporale o spaziale inferiore. Ad esempio, l'ordinamento a bolle (complessità $O(n^2)$) potrebbe essere sostituito con l'ordinamento QuickSort (complessità media O(n log n)).

Modulo 3

Ottimizzazione di algoritmi sequenziali

Ottimizzazione delle Strutture Dati

•**Scelta Appropriata delle Strutture Dati:** Utilizzare la struttura dati più adatta può ridurre significativamente il tempo di esecuzione. Per esempio, l'uso di hashtable per operazioni di ricerca rapida rispetto a una lista.

•**Efficienza della Memoria:** Ridurre l'utilizzo della memoria, ad esempio evitando la duplicazione inutile dei dati.

Ottimizzazione del Codice

•**Riduzione delle Istruzioni nei Cicli:** Minimizzare il numero di operazioni all'interno di cicli critici.

•**Evitare Calcoli Ridondanti:** Memorizzare i risultati di calcoli ripetuti, come nell'uso della memoizzazione in programmazione dinamica.

•**Uso di Algoritmi In Loco:** Quando possibile, modificare i dati in loco per ridurre l'uso della memoria.

Modulo 3

Ottimizzazione di algoritmi sequenziali

Uso di Tecniche Specifiche

•**Euristica e Algoritmi Approssimativi:** In alcuni problemi complessi, un'euristica o un algoritmo approssimativo può fornire una soluzione abbastanza buona in meno tempo.

•**Parallelizzazione:** Per i sistemi che lo consentono, dividere il lavoro in parti che possono essere eseguite in parallelo.

Profiling e Testing

•**Profiling del Codice:** Utilizzare strumenti di profiling per identificare colli di bottiglia nel codice.

•**Testing Rigoroso:** Assicurarsi che le ottimizzazioni non compromettano la correttezza del programma.

Modulo 3

Ottimizzazione di algoritmi sequenziali

Ricerca Continua e Aggiornamenti

•**Mantenere il Codice Aggiornato:** Con l'evoluzione delle piattaforme hardware e delle tecniche di programmazione, è utile rivedere periodicamente gli algoritmi.

•**Essere Informati sulle Novità:** Restare aggiornati sulle ultime ricerche e tendenze nel campo degli algoritmi può offrire nuove idee per ottimizzazioni.

L'ottimizzazione di algoritmi è un bilanciamento tra vari fattori, come velocità, uso della memoria, e complessità del codice. È importante considerare le esigenze specifiche dell'applicazione e scegliere la strategia di ottimizzazione più appropriata. In molti casi, un'ottimizzazione significativa può essere raggiunta senza complicare eccessivamente il codice, mantenendo così un buon equilibrio tra prestazioni e manutenibilità.

Modulo 3

Debugging e identificazione degli errori

Il debugging è un aspetto cruciale dello sviluppo del software, che comporta il processo di identificazione e risoluzione di errori ("bug") nel codice. Un approccio metodico e sistematico al debugging è essenziale per risolvere efficacemente i problemi che emergono durante lo sviluppo e il test del software.

Modulo 3

Debugging e identificazione degli errori

Tipi di Errori

In generale, gli errori nel codice possono essere classificati in diverse categorie:

1.Errori di Sintassi: Sono errori relativi alla struttura del linguaggio di programmazione, come l'omissione di parentesi, l'uso errato di identificatori, o la mancata chiusura di stringhe o blocchi di codice.

2.Errori di Runtime: Si verificano quando il programma è in esecuzione e possono includere cose come divisione per zero, accesso a indici di array fuori range, o chiamate a funzioni con argomenti errati.

3.Errori Logici: Sono tra i più difficili da individuare e risolvere. Si verificano quando il programma viene eseguito senza crash, ma produce un output non corretto o inatteso a causa di una logica errata nel codice.

Modulo 3

Debugging e identificazione degli errori

Strategie di Debugging

Il processo di debugging può variare in base alla natura del problema, ma alcune strategie comuni includono:

1. **Analisi dei Messaggi di Errore:** Spesso, i messaggi di errore forniti dai compilatori o dagli ambienti di runtime possono indicare la posizione o la natura dell'errore.
2. **Stampa di Debug e Logging:** Utilizzare istruzioni di stampa o strumenti di logging per tracciare il flusso di esecuzione e i valori delle variabili.
3. **Debugger:** Utilizzare strumenti di debugging che permettono di eseguire il programma passo dopo passo, esaminare lo stato delle variabili, e fermarsi in punti specifici del codice (breakpoints).
4. **Test Unitari e Test di Integrazione:** Scrivere test per parti specifiche del codice può aiutare a isolare e identificare dove si verificano gli errori.
5. **Code Review:** A volte, un altro paio di occhi può individuare errori che si potrebbero aver trascurato. La revisione del codice da parte di colleghi può essere molto utile.
6. **Riflessione e Ricerca:** Se un problema sembra insormontabile, a volte può essere utile fare una pausa e riflettere, o cercare soluzioni online (ad esempio, su Stack Overflow).
7. **Semplificazione del Codice:** Se il bug è difficile da isolare, provare a semplificare o decomporre il codice per isolare la parte problematica.

Modulo 3

Debugging e identificazione degli errori

Buone Pratiche

•**Scrivere Codice Pulito e Manutenibile:** Un codice ben organizzato e leggibile è più facile da debuggare.

•**Commentare il Codice:** I commenti possono aiutare a comprendere cosa dovrebbe fare una determinata sezione di codice.

•**Adottare un Approccio Incrementale:** Aggiungere e testare piccole porzioni di codice alla volta può aiutare a prevenire errori complessi.

•**Version Control:** Utilizzare sistemi di controllo versione come Git per tenere traccia delle modifiche e facilitare il rollback a versioni precedenti se necessario.

Il debugging è un'abilità essenziale per qualsiasi sviluppatore di software e richiede pazienza, attenzione ai dettagli e una buona comprensione del codice e delle tecniche di programmazione.

Modulo 3

Test e verifica di algoritmi

Il testing degli algoritmi implica eseguire il codice con vari set di dati di input per verificare che l'output sia quello atteso. Ci sono diversi tipi di test che possono essere eseguiti:

1. **Test Unitario:** Verifica le singole parti (unità) di un algoritmo, come funzioni o metodi, per assicurarsi che funzionino correttamente in modo isolato. Gli sviluppatori scrivono di solito test unitari per ogni funzione importante.
2. **Test di Integrazione:** Si concentra sulle interazioni tra diverse unità o componenti dell'algoritmo per assicurarsi che lavorino insieme come previsto.
3. **Test di Sistema:** Testa l'algoritmo nel suo insieme per assicurarsi che soddisfi tutti i requisiti e le specifiche.
4. **Test di Regressione:** Assicura che le nuove modifiche o aggiunte al codice non abbiano introdotto nuovi errori in parti già testate.
5. **Test di Accettazione:** Spesso eseguito dal cliente, questo test verifica se l'algoritmo soddisfa i requisiti e le aspettative del cliente.

Modulo 3

Test e verifica di algoritmi

Verifica di Algoritmi

La verifica, invece, è un processo che si concentra sulla correttezza logica dell'algoritmo. Include:

1. **Verifica Formale:** Si utilizzano metodi matematici e logici per dimostrare che l'algoritmo rispetta determinate proprietà e specifiche. Questo può includere la verifica di loop, la validità di algoritmi ricorsivi, ecc.
2. **Revisione del Codice:** Processo in cui il codice viene esaminato da altri sviluppatori per identificare errori, problemi di logica e potenziali miglioramenti.
3. **Analisi Statica:** Utilizza strumenti software per esaminare il codice sorgente e rilevare problemi come errori di sintassi, violazioni di stile, errori logici e potenziali vulnerabilità senza eseguire effettivamente il programma.

Modulo 3

Test e verifica di algoritmi

Best Practices

•**Automazione dei Test:** Utilizzare framework di testing come JUnit (Java), PyTest (Python), o altri strumenti simili per scrivere e eseguire automaticamente i test.

•**Copertura dei Test:** Assicurarsi che i test coprano una vasta gamma di scenari, inclusi casi limite e comportamenti anomali.

•**Testing Continuo:** Integrare il testing nel processo di sviluppo software, eseguendolo regolarmente durante tutto il ciclo di vita dello sviluppo.

•**Documentazione dei Test:** Mantenere una documentazione chiara dei test eseguiti e dei loro risultati.

Testare e verificare gli algoritmi non solo aiuta a individuare e risolvere problemi specifici, ma migliora anche la comprensione generale del funzionamento dell'algoritmo e della sua interazione con il resto del sistema. Questi processi sono essenziali per lo sviluppo di software di qualità e affidabile.

Modulo 3

Introduzione alla complessità algoritmica

La complessità algoritmica è un concetto fondamentale in informatica che riguarda la valutazione dell'efficienza degli algoritmi in termini di risorse utilizzate, come il tempo di esecuzione e lo spazio di memoria. Questo concetto è cruciale per comprendere come le prestazioni di un algoritmo variano al cambiare delle dimensioni dell'input.

Definizione

La complessità algoritmica è misurata considerando due aspetti principali:

- **Complessità Temporale:** Quantifica il tempo necessario per eseguire un algoritmo in funzione della dimensione dell'input. È spesso espressa come una funzione del numero di operazioni eseguite.
- **Complessità Spaziale:** Misura la quantità di memoria richiesta dall'algoritmo, anch'essa in funzione della dimensione dell'input.

Modulo 3

Introduzione alla complessità algoritmica

Analisi della Complessità

•**Notazione Big O:** È il modo più comune per descrivere la complessità di un algoritmo. Indica il limite superiore della crescita della funzione che rappresenta la complessità. Ad esempio, O(n) indica che il tempo di esecuzione aumenta linearmente con la dimensione dell'input, mentre $O(n^2)$ indica una crescita quadratica.

•**Notazione Big Omega (Ω):** Indica il limite inferiore della complessità algoritmica.

•**Notazione Big Theta (Θ):** Fornisce una stretta banda di crescita, indicando sia il limite inferiore sia quello superiore.

Complessità Temporale

Esempi comuni includono:

•**Costante (O(1)):** Il tempo di esecuzione non dipende dalla dimensione dell'input.

•**Lineare (O(n)):** Il tempo di esecuzione cresce linearmente con l'incremento della dimensione dell'input.

•**Quadratica ($O(n^2)$):** Il tempo di esecuzione cresce quadraticamente all'aumentare dell'input.

•**Logaritmica (O(log n)):** Il tempo di esecuzione aumenta logaritmicamente con l'aumento dell'input, tipico di algoritmi efficienti come la ricerca binaria.

Modulo 3

Introduzione alla complessità algoritmica

Complessità Spaziale

•Dipende da quanto spazio di memoria l'algoritmo richiede. Ad esempio, un algoritmo che usa una matrice di dimensioni n x n avrà una complessità spaziale $O(n^2)$.

Importanza

La complessità algoritmica è fondamentale per:

•**Valutare l'Efficienza:** Permette di confrontare l'efficienza di diversi algoritmi per lo stesso problema.

•**Prevedere le Prestazioni:** Aiuta a prevedere come le prestazioni cambieranno con grandi volumi di dati.

•**Sviluppo di Algoritmi Efficaci:** Incoraggia lo sviluppo di algoritmi che funzionano bene anche con grandi quantità di dati.

Comprendere la complessità algoritmica è essenziale per la progettazione di software efficiente e performante, specialmente in contesti dove le risorse come il tempo di calcolo e la memoria sono limitate.

Modulo 3

Esempi di codice sequenziale

1. Calcolo della Somma di una Lista di Numeri

Questo esempio calcola la somma di tutti i numeri in una lista.

```python
def calcola_somma(numeri):
    somma = 0
    for num in numeri:
        somma += num
    return somma

numeri = [1, 2, 3, 4, 5]
print("Somma:", calcola_somma(numeri))
```

Modulo 3

Esempi di codice sequenziale

2. Ricerca del Massimo in una Lista

Questo frammento trova il valore massimo in una lista.

```
def trova_massimo(numeri):
    massimo = numeri[0]
    for num in numeri:
        if num > massimo:
            massimo = num
    return massimo

numeri = [3, 6, 2, 8, 4, 7]
print("Massimo:", trova_massimo(numeri))
```

Modulo 3

Esempi di codice sequenziale

3. Ordinamento a Bolle

Un semplice algoritmo di ordinamento che confronta elementi adiacenti e li scambia se sono nell'ordine sbagliato.

```
def bubble_sort(numeri):
    n = len(numeri)
    for i in range(n):
        for j in range(0, n-i-1):
            if numeri[j] > numeri[j+1]:
                numeri[j], numeri[j+1] =
numeri[j+1], numeri[j]

numeri = [64, 34, 25, 12, 22, 11, 90]
bubble_sort(numeri)
print("Ordinati:", numeri)
```

Modulo 3

Esempi di codice sequenziale

4. Calcolo del Fattoriale

Calcolo del fattoriale di un numero usando un approccio iterativo.

```
def fattoriale(n):
    risultato = 1
    for i in range(2, n+1):
        risultato *= i
    return risultato

n = 5
print("Fattoriale di", n, ":", fattoriale(n))
```

Modulo 3

Esempi di codice sequenziale

5. Stampa dei Numeri Fibonacci

Stampa i primi n numeri della sequenza di Fibonacci.

```
def fibonacci(n):
    a, b = 0, 1
    for i in range(n):
        print(a, end=" ")
        a, b = b, a + b

fibonacci(10)
```

Ognuno di questi esempi mostra come si possa eseguire un'operazione o calcolo in una sequenza di passi logici. Sono tipici esempi di programmazione procedurale dove ogni istruzione viene eseguita una dopo l'altra.

Modulo 3

OOP

Introduzione alla OOP: concetti chiave

L'Object-Oriented Programming (OOP), o programmazione orientata agli oggetti, è un paradigma di programmazione basato sull'idea di "oggetti", che possono contenere dati (in forma di attributi) e codice (in forma di metodi). Python è un linguaggio che supporta pienamente l'OOP e lo rende semplice ed efficace. Di seguito sono riportati alcuni concetti chiave della OOP in Python:

1. Classi e Oggetti

•**Classe:** Una classe è un modello o un prototipo da cui gli oggetti sono creati. Definisce un tipo di dati, racchiudendo dati (attributi) e funzioni (metodi) che lavorano su questi dati.

•**Oggetto:** Un oggetto è un'istanza di una classe. Se una classe è un "disegno", un oggetto è una "casa" costruita seguendo quel disegno.

```
class Automobile:
    def_init_(self, marca, modello):
        self.marca  =  marca
        self.modello = modello

    def mostra_dettagli(self):
        print(f"Marca: {self.marca}, Modello:
{self.modello}")

# Creazione di un oggetto
mia_auto = Automobile("Toyota", "Corolla")
mia_auto.mostra_dettagli()
```

Modulo 3

OOP

Incapsulamento

L'incapsulamento è il concetto di nascondere i dettagli interni di un oggetto e mostrare solo le funzionalità necessarie all'esterno. In Python, l'incapsulamento viene realizzato utilizzando metodi privati e attributi (generalmente prefissati con _ o___).

Modulo 3

OOP

Ereditarietà

L'ereditarietà permette di definire una nuova classe che acquisisce, o eredita, le proprietà di una classe esistente. Facilita il riutilizzo del codice e la realizzazione di gerarchie di classi.

```
class Veicolo:
    def__init__(self, categoria):
        self.categoria = categoria

class Moto(Veicolo):
    def__init__(self, categoria, ruote):
        super()._init_(categoria)
        self.ruote = ruote

moto = Moto("Motocicletta", 2)
print(f"Categoria: {moto.categoria}, Ruote: {moto.ruote}")
```

Modulo 3

OOP

Polimorfismo

Il polimorfismo si riferisce alla possibilità di utilizzare un'interfaccia comune per diversi tipi di dati. In Python, il polimorfismo si manifesta tipicamente attraverso l'overriding dei metodi (quando una classe figlia fornisce una specifica implementazione di un metodo già definito nella sua classe genitore) o l'overloading dei metodi (che Python supporta in una forma limitata rispetto ad altri linguaggi).

```
class Animale:
    def parla(self):
        pass

class Cane(Animale):
    def parla(self):
        return "Bau!"

class Gatto(Animale):
    def parla(self):
        return "Miao!"

animale  =  Cane()
print(animale.parla())  # Outputs: Bau!
```

Modulo 3

OOP

Astrazione

L'astrazione si concentra sull'idea di nascondere la complessità e mostrare solo il necessario. In Python, le classi astratte forniscono un modello per altre classi che derivano da esse e possono contenere metodi astratti che devono essere implementati dalle classi figlie. Questi concetti formano la base della OOP in Python, permettendo agli sviluppatori di creare codice riutilizzabile, organizzato e modulare.

Modulo 3

OOP

Classi astratte e interfacce

In Python, le classi astratte e le interfacce sono utilizzate per definire una struttura di base che altre classi possono ereditare e implementare. Vediamo in dettaglio questi concetti:

Modulo 3

OOP

Classi astratte e interfacce

Classi Astratte

Una classe astratta è una classe che non può essere istanziata da sola e che è destinata a essere sottoclasse. Viene utilizzata per definire un template per altre classi. Python fornisce il modulo **abc** (Abstract Base Classes) per lavorare con classi astratte.
Ecco come si utilizza:

1. **Importare ABC e abstractmethod:** Dal modulo **abc**, importa **ABC** e **abstractmethod**. **ABC** sta per Abstract Base Class e **abstractmethod** è un decoratore per indicare metodi astratti.
2. **Definire una Classe Astratta:** Una classe astratta eredita da **ABC** non può essere istanziata da sola e solitamente contiene uno o più metodi astratti.
3. **Metodi Astratti:** Un metodo astratto è un metodo che ha una dichiarazione ma nessuna implementazione. Le classi che ereditano la classe astratta devono implementare questi metodi.

```python
from abc import ABC, abstractmethod

class Animale(ABC):
    @abstractmethod
    def emetti_suono(self):
        pass

class Cane(Animale):
    def emetti_suono(self):
        return "Bau!"

# animale = Animale() # Errore: non si possono istanziare classi astratte
cane = Cane()
print(cane.emetti_suono())  # Stampa "Bau!"
```

Modulo 3

OOP Classi astratte e interfacce

Interfacce

In Python, il concetto di "interfaccia" non è definito in modo rigoroso come in altri linguaggi come Java. Tuttavia, possiamo utilizzare le classi astratte per simulare il comportamento delle interfacce. In questo contesto, un'interfaccia è una classe astratta in cui tutti i metodi sono astratti.
Python segue il principio di "duck typing", dove l'interoperabilità di un oggetto è determinata dai metodi e dagli attributi che esso implementa, piuttosto che dall'ereditarietà da una classe specifica.
Quindi, in pratica, un'interfaccia in Python è più una guida o un contratto che definisce quali metodi una classe dovrebbe implementare.

```
class Interruttore(ABC):
    @abstractmethod
    def accendi(self):
        pass

    @abstractmethod
    def spegni(self):
        pass

class Lampadina(Interruttore):
    def accendi(self):
        print("Lampadina accesa")

    def spegni(self):
        print("Lampadina spenta")

lampadina = Lampadina()
lampadina.accendi()  # Stampa "Lampadina accesa"
lampadina.spegni()   # Stampa "Lampadina spenta"
```

In questo esempio, Interruttore agisce come un'interfaccia che definisce un contratto per le classi Lampadina. La classe Lampadina implementa questa interfaccia fornendo un comportamento concreto per i metodi accendi e spegni.

Modulo 3

OOP :: Composizione e aggregazione in python

In Python, così come in altri linguaggi di programmazione orientati agli oggetti, la composizione e l'aggregazione sono due modi di stabilire relazioni tra oggetti. Entrambe sono forme di associazione che definiscono le relazioni "ha-un" tra oggetti, ma differiscono nel grado di dipendenza tra le classi coinvolte.

Composizione

La composizione è una relazione "ha-un" strettamente accoppiata, dove una classe (detta "compositore") contiene un oggetto di un'altra classe (detto "componente") come parte della sua struttura. La composizione implica che quando il compositore viene distrutto, anche i suoi componenti vengono distrutti.

Caratteristiche della Composizione:

- Forte relazione di proprietà.
- Il ciclo di vita degli oggetti componenti è legato al ciclo di vita dell'oggetto compositore.
- Ad esempio, un oggetto "Auto" può contenere oggetti "Motore", "Ruota", ecc., e se l'auto viene distrutta, anche il motore e le ruote cesseranno di esistere.

Composizione e aggregazione in python

```
class Motore:
    def___init__(self, potenza):
        self.potenza = potenza

class Auto:
    def_init_(self, modello, potenza_motore):
        self.modello = modello
        self.motore = Motore(potenza_motore)

    def mostra_dettagli(self):
        print(f"Auto modello {self.modello} con motore di potenza
{self.motore.potenza}")

auto = Auto("Ferrari", "500cv")
auto.mostra_dettagli()
```

Modulo 3

OOP :: Composizione e aggregazione in python

Aggregazione

L'aggregazione è una forma più debole di relazione "ha-un", dove il compositore contiene oggetti di altre classi, ma la loro esistenza non dipende strettamente dal compositore.

Caratteristiche dell'Aggregazione:

•Relazione meno stretta rispetto alla composizione.

•Gli oggetti componenti possono sopravvivere indipendentemente dal compositore.

•Ad esempio, una classe "Squadra" può contenere oggetti "Giocatore", ma i giocatori possono esistere anche al di fuori della squadra.

```
class Giocatore:
    def __init__(self, nome):
        self.nome = nome

class Squadra:
    def __init__(self, nome):
        self.nome = nome
        self.giocatori = []

    def aggiungi_giocatore(self, giocatore):
        self.giocatori.append(giocatore)

    def mostra_giocatori(self):
        for giocatore in self.giocatori:
            print(giocatore.nome)

giocatore1 = Giocatore("Mario")
giocatore2 = Giocatore("Luigi")
squadra = Squadra("Stelle")
squadra.aggiungi_giocatore(giocatore1)
squadra.aggiungi_giocatore(giocatore2)

squadra.mostra_giocatori()
```

Modulo 3

OOP :: Composizione e aggregazione in python

In questo esempio, la Squadra contiene Giocatori, ma i Giocatori possono esistere anche al di fuori della Squadra.

Riassunto

•**Composizione:** Relazione "ha-un" con dipendenza forte e condivisa del ciclo di vita.
•**Aggregazione:** Relazione "ha-un" con dipendenza più debole e ciclo di vita indipendente.
Queste relazioni sono fondamentali per creare strutture di oggetti complesse in Python e in altri linguaggi OOP, consentendo una progettazione del software più modulare e flessibile.

Modulo 3

OOP :: Designi pattern

I design pattern nell'Object-Oriented Programming (OOP) sono soluzioni consolidate a problemi comuni di progettazione software. Sono come modelli che possono essere adattati ai bisogni specifici di un progetto per risolvere determinate questioni in modo efficace.

I più comuni

Di seguito vediamo i più comuni.

1. Singleton

Questo pattern assicura che una classe abbia una sola istanza e fornisce un punto di accesso globale a quella istanza. È spesso utilizzato per gestire risorse condivise, come connessioni al database.

```
class Singleton:
    _instance = None

    @staticmethod
    def getInstance():
        if Singleton._instance == None:
            Singleton()
        return Singleton._instance

    def __init__(self):
```

Modulo 3

OOP :: Designi pattern

1. Singleton

Questo pattern assicura che una classe abbia una sola istanza e fornisce un punto di accesso globale a quella istanza. È spesso utilizzato per gestire risorse condivise, come connessioni al database.

```
class Singleton:
    _instance = None

    @staticmethod
    def getInstance():
        if Singleton._instance == None:
            Singleton()
        return Singleton._instance

    def__init__(self):
        if Singleton._instance != None:
            raise Exception("Questa classe è un singleton!")
        else:
            Singleton._instance = self
```

Modulo 3

OOP :: Designi pattern

2. Factory Method

Il Factory Method fornisce un'interfaccia per creare oggetti in una superclasse, ma permette alle sottoclassi di modificare il tipo di oggetti che verranno creati.

```
class Automobile:
    def factory(tipo):
        if tipo == "Sportiva":
            return Sportiva()
        if tipo == "Familiare":
            return Familiare()
        raise Exception("Tipo di automobile non trovato")

    factory = staticmethod(factory)

class Sportiva(Automobile):
    pass

class Familiare(Automobile):
    pass

# Uso
auto = Automobile.factory("Sportiva")
```

Modulo 3

OOP :: Designi pattern

3. Observer

In questo pattern, un oggetto, noto come "subject", mantiene una lista di dipendenze ("observers") e li notifica automaticamente di eventuali cambiamenti di stato.

```
class Subject:
    def_init_(self):
        self._observers = []

    def attach(self, observer):
        self._observers.append(observer)

    def notify(self):
        for observer in self._observers:
            observer.update(self)

class Observer:
    def update(self, subject):
        pass
```

Modulo 3

OOP :: Designi pattern

4. Strategy

Questo pattern consente di definire una famiglia di algoritmi, incapsularli come oggetti e renderli intercambiabili. La Strategy permette all'algoritmo di variare indipendentemente dai client che lo utilizzano.

```
class Strategia:
    def esegui(self):
        pass

class ConcreteStrategiaA(Strategia):
    def esegui(self):
        print("Esecuzione strategia A")

class ConcreteStrategiaB(Strategia):
    def esegui(self):
        print("Esecuzione strategia B")

class Context:
    def_init_(self, strategia):
        self._strategia = strategia

    def esegui_strategia(self):
        self._strategia.esegui()

# Uso
strategia = ConcreteStrategiaA()
context = Context(strategia)
context.esegui_strategia()
```

Modulo 3

OOP :: Designi pattern

5. Decorator

Decorator aggiunge dinamicamente responsabilità aggiuntive a un oggetto. Questi pattern forniscono un'alternativa flessibile all'ereditarietà per estendere le funzionalità.

```
class Componente:
    def operazione(self):
        pass

class ConcreteComponente(Componente):
    def operazione(self):
        print("Operazione base")

class Decoratore(Componente):
    def_init_(self, componente):
        self._componente = componente

    def operazione(self):
        self._componente.operazione()

class ConcreteDecoratoreA(Decoratore):
    def operazione(self):
        super().operazione()
        print("Operazione estesa da Decoratore A")

# Uso
componente = ConcreteComponente()
decoratore = ConcreteDecoratoreA(componente)
decoratore.operazione()
```

Modulo 3

OOP :: Designi pattern

Ognuno di questi pattern risolve problemi specifici nella progettazione software e può essere adattato a diversi scenari. Saper riconoscere e applicare questi pattern può aiutare a scrivere codice più pulito, organizzato e riutilizzabile.

Modulo 3

OOP :: Proprietà e metodi

In Python, le proprietà e i metodi speciali sono elementi fondamentali della programmazione orientata agli oggetti che migliorano la flessibilità e l'efficienza del codice. Essi permettono agli sviluppatori di creare interfacce più pulite e di integrare più strettamente le proprie classi e oggetti con il linguaggio Python.

Proprietà

Le proprietà in Python sono un modo di accedere ai membri privati di una classe. Forniscono un'interfaccia per accedere agli attributi di un oggetto, consentendo di definire getter, setter e deleter, che sono metodi utilizzati per ottenere, impostare o eliminare il valore di un attributo.

```python
class Persona:
    def __init__(self):
        self._nome = ""

    @property
    def nome(self):
        return self._nome

    @nome.setter
    def nome(self, valore):
        self._nome = valore

    @nome.deleter
    def nome(self):
        del self._nome

persona = Persona()
persona.nome = "Alice"
print(persona.nome)
del persona.nome
```

Modulo 3

OOP :: Proprietà e metodi

Metodi Speciali

I metodi speciali in Python sono spesso definiti come "metodi magici" e sono caratterizzati da doppi underscore (__) all'inizio e alla fine del loro nome. Sono utilizzati per sovrascrivere o estendere il comportamento predefinito degli oggetti Python. Alcuni esempi comuni includono:

- **__init__(self, ...)**: Il costruttore della classe, chiamato quando un nuovo oggetto viene creato.
- **__str__(self)**: Restituisce la rappresentazione dell'oggetto come stringa, utilizzato da **print()** e **str()**.
- **__repr__(self)**: Restituisce una rappresentazione ufficiale dell'oggetto, spesso utilizzata per debugging.
- **__len__(self)**: Restituisce la lunghezza dell'oggetto, chiamato dalla funzione **len()**.
- **__getitem__(self, key)**: Permette di accedere agli elementi dell'oggetto usando l'indicizzazione come **obj[key]**.
- **__eq__(self, other)**: Definisce il comportamento per l'operatore di uguaglianza, **==**.
- **__add__(self, other)**: Definisce il comportamento per l'operatore di addizione, **+**.

```python
class Libro:
    def_init_(self, titolo, autore):
        self.titolo = titolo
        self.autore = autore

    def___str__(self):
        return f"{self.titolo} di {self.autore}"

libro = Libro("Il Nome della Rosa", "Umberto Eco")
print(libro)  # Chiama___str___
```

Modulo 3

OOP :: Metaclassi e decoratori

L'Object-Oriented Programming (OOP) in Python include concetti avanzati come metaclassi e decoratori, che offrono un livello più elevato di controllo e flessibilità nel modo in cui il codice viene scritto e gestito.

Modulo 3

OOP :: Metaclassi e decoratori

Metaclassi

Le metaclassi sono "classi di classi", ovvero definiscono come le classi vengono create. Sono un livello sopra le classi normali in Python e permettono di intervenire nella creazione delle classi.

Caratteristiche delle Metaclassi:

- **Controllo della Creazione di Classi:** Le metaclassi possono modificare o estendere la creazione di una classe.
- **Interfaccia Avanzata:** Sono utilizzate per creare interfacce complesse o frameworks.

Sovrascrittura di type: In Python, **type** è la metaclassa predefinita che Python usa per creare tutte le classi.

```
class Meta(type):
    def __new__(cls, name, bases, dct):
        # custom actions...
        return super().__new__(cls, name, bases, dct)

class MyClass(metaclass=Meta):
    pass
```

Modulo 3

OOP :: Metaclassi e decoratori

Decoratori

I decoratori sono un potente strumento in Python che permette di modificare il comportamento di funzioni e metodi. Sono spesso usati per aggiungere funzionalità a funzioni esistenti in un modo pulito e leggibile.

Caratteristiche dei Decoratori:

- **Modificare o Estendere Funzioni:** Aggiungono o modificano il comportamento di una funzione o metodo senza modificarne il codice sorgente.
- **Riuso del Codice:** Possono essere applicati a diverse funzioni per evitare la duplicazione del codice.
- **Sintassi Semplice:** Utilizzano il simbolo **@** seguito dal nome del decoratore.

```python
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("Qualcosa eseguito prima della funzione.")
        result = func(*args, **kwargs)
        print("Qualcosa eseguito dopo la funzione.")
        return result
    return wrapper

@my_decorator
def saluta(nome):
    print(f"Ciao {nome}!")

saluta("Alice")
```

Modulo 3

OOP :: Metaclassi e decoratori

```python
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("Qualcosa eseguito prima della funzione.")
        result = func(*args, **kwargs)
        print("Qualcosa eseguito dopo la funzione.")
        return result
    return wrapper

@my_decorator
def saluta(nome):
    print(f"Ciao {nome}!")

saluta("Alice")
```

In questo esempio, **my_decorator** è un decoratore che esegue un codice prima e dopo la funzione **saluta**.

Riassunto

- **Metaclassi:** Strumenti avanzati per gestire la creazione di classi, utilizzati per costruire frameworks o interfacce complesse.
- **Decoratori:** Forniscono un modo semplice per estendere o modificare il comportamento di funzioni o metodi esistenti, aiutando a mantenere il codice pulito e riducendo la duplicazione.

Entrambi questi concetti sono avanzati e possono essere molto potenti quando usati correttamente, ma è anche importante usarli con cautela per mantenere il codice comprensibile e manutenibile.